Lab 7

Objective: Program in 6809 assembly
This lab builds on lectures 5, 6, and 7

Background
In this lab we will learn 6809 assembly. The 6809 is quite similar to the 4-bit CPU we used in the previous lab, except that we have more registers, more instructions, and more memory (65,536 bytes instead of 16 bytes).

Exercise 1: Familiarize yourself with the simulator
Let's use the simulator. We will enter the program below, run the assembler, and watch the stack, as we single step through the program. 6809 simulator: https://cosc204.cspages.otago.ac.nz/online6809/

a) Enter the following program; comments are green*

```
SCREEN EQU $0400                ; start of screen

START:
    LDY #MESSAGE                ; get the address of the string
    BSR PUTS                    ; call puts to print it
    RTS                         ; return to the OS
PUTS:
    PSHS A,X,Y                  ; save a, x, and y
    LDX #SCREEN                 ; start of screen
MORE:
    LDA 0,Y+                    ; current char -> A (inc Y too)
    CMPA #$00                   ; was it a zero?
    BEQ DONE                    ; if it was 0 then return
    STA 0,X+                    ; write (inc X too)
    BRA MORE                    ; repeat
DONE:
    PULS A,X,Y                  ; restore a, x, y
    RTS                         ; return from this routine
MESSAGE:
    .BYTE "HELLO WORLD", $00    ; the message to display"
```

*Syntax highlighting with Visual Studio Code extension "6x09 Assembly"

b) Click **`Assemble source code`**
You just ran the assembler to convert the assembly instructions into machine code. Much easier than doing it by hand! The `Disassembly output` window shows where, in memory, your program is stored ($4000), the machine code, and the disassembly (the machine code turned back into assembly language).

c) Examine the registers
The 6809 has several registers: A, B, X, Y, U, S, CC, DP, PC (D is A and B combined).
What is the value of the system stack pointer S?
Remember, *pointer* just means "an index into the array of memory".

d) Examine the stack
Underneath the disassembly the simulator shows a small window into the memory of the computer. The column starting 7FF0 has 16 bytes, $7FF0-$7FFF. This includes the part of the memory that has the stack in it. Why does this end F0 18?

e) Examine the **`Text screen`**
Below the disassembly window is the *text screen*. It is *memory mapped* to location $0400. That means anything written to memory page $04 or $05 appear on the screen. Run your program and watch the text screen change.

f) Single step
Save your program by copying it into a text editor and saving it. Refresh the simulator website. Paste your program back in and assemble. This will reset the computer and clear the text screen. Now `Step` the program one instruction at a time and watch the registers change, watch the memory holding the stack change, and watch the text screen change.
When PUTS is called what happens?
When PUTS finishes with the `RTS` instruction, what happens?

Exercise 2: Looping
Assembly does not have high-level loop constructs like `for`, `while`, `do`... It does have comparison instructions (`CMPA`, `CMPX`, etc.) and conditional branch instructions (`BEQ`, `BGE`, etc.). You can use the instructions to create loops and `if` statements. We're going to write a loop that clears the simulator's `Text screen`. We know the screen starts at location $0400, and we know that the screen occupies 512 bytes ($0200 bytes) of memory because it is 16 lines of 32 characters. So we want a loop, like a `for` loop, that writes a space character (#$20) to the 512 bytes starting at location $0400. Such a for loop in C might look like this (where 0x in C means hex number):

```
for (uint32_t x = 0x0400; x != 0x0600; x++)
   memory[x] = 0x20;
```

In 6809 assembly its nearly the same. First assign $0400 to X, then compare X to $0600, if it is equal then exit the loop, else write to memory location X and increment X.

```
START:
  LDA #$20          ; the space character
  LDX #$0400        ; start of the screen
MORE:
  CMPX #$0600       ; the for loop comparison (compare x to $600)
  BEQ DONE          ; if equal then branch to done (exit the loop)
  STA 0,X+          ; otherwise do the body of the loop (inc X too)
  BRA MORE          ; continue the loop
DONE:
  RTS               ; after the loop
```

Write your own loop to add the numbers between 4 and 9 (including 4 and 9).
Hint: you can use `B` as the loop variable and accumulate into `X` using the `ABX` instruction. The numerically correct answer is 39 ($27).

Exercise 3: Memory as variables.
In high level languages (like C) we declare variables and use them by referring to their names.
The compiler turns accesses to these variables into memory load and memory store instructions.
In 6809 assembly we have to do this manually. For example, in C we can write:

```
uint16_t var1 = 0xBEEF;
uint16_t var2 = 0x1FBE;
uint16_t var3 = 0x0000;

var3 = var1 + var2;
```

But in 6809 assembly we must do this:

```
START:
  LDD VAR1                  ; load D with VAR1 (D = VAR1)
  ADDD VAR2                 ; add VAR2 (D += VAR2)
  STD VAR3                  ; store back into VAR3 (VAR3 = D)
  RTS

VAR1:                       ; 16-bit value (uint16_t VAR1 = $BEEF)
  .WORD $BEEF
VAR2:                       ; alternative way (2 bytes is 1 word)
  .BYTE $1F, $BE            ; uint16_t VAR2 = $1FBE
VAR3:                       ; uint16_t VAR3 = 0
  .WORD $0000
```

a) Write a program in 6809 assembly language that swaps the value of two variables `VAR1`, and `VAR2`.
Hint: in C or Java you would declare a third variable (`tmp`). Can you do the same in 6809 assembly?

b) Re-write this program so that it doesn't use a temporary variable.
Hint: you have more than 1 register.